



High-Performance K-Means

Optimized From-Scratch Implementation

Munusamy M (ZDA25M011)

Programming and Data Structures

Instructor: Dr. Innocent Nyalala

IIT Madras Zanzibar

Motivation

Why Optimize K-Means?

- ▶ K-Means is simple but computationally expensive
- ▶ Naive loop-based implementations do not scale
- ▶ Distance computation dominates runtime

Project Objectives

Goals

- ▶ Implement K-Means completely from scratch
- ▶ Avoid external ML libraries
- ▶ Use efficient data structures
- ▶ Achieve performance comparable to scikit-learn

System Design Overview

Conceptual Pipeline

- ▶ Data loading, cleaning, and standardization
- ▶ K-Means++ for centroid initialization
- ▶ Lloyd's iterative clustering algorithm
- ▶ Evaluation using WCSS and Elbow Method
- ▶ Benchmarking against scikit-learn

Vectorized Distance Computation

Key Optimization

$$\|x - c\|^2 = \|x\|^2 + \|c\|^2 - 2x \cdot c$$

- ▶ Fully vectorized using NumPy
- ▶ Eliminates nested loops
- ▶ Major performance improvement

K-Means++ Initialization

Why K-Means++?

- ▶ Better initial centroid placement
- ▶ Faster convergence
- ▶ Reduced sensitivity to local minima

Custom Data Structures

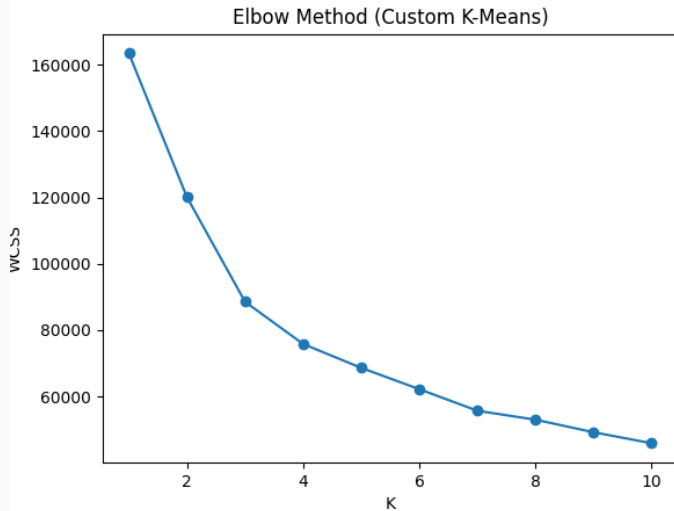
Binary Min-Heap

- ▶ Used in K-Means++ initialization
- ▶ Used for empty cluster recovery

Hash Table

- ▶ Stores WCSS values for Elbow Method

Elbow Method Result



Optimal number of clusters observed around $K = 4-5$

Benchmarking Results

Custom vs scikit-learn ($n_{init} = 1$)

- ▶ Dataset: 20,433 samples, 8 features
- ▶ WCSS Error: 1.43%
- ▶ Runtime: 0.56s (Custom) vs 4.32s (scikit-learn)

Complexity Analysis

Asymptotic Complexity

- ▶ Distance computation: $O(NKF)$
- ▶ Assignment step: $O(NK)$
- ▶ Overall complexity: $O(TNKF)$

Conclusion

Key Takeaways

- ▶ High-performance K-Means built from scratch
- ▶ Vectorization yields significant speed-up
- ▶ Custom data structures enhance robustness
- ▶ Results comparable to standard libraries

Thank You

Questions?